

Week01 | 实验 | 项目基线与系统蓝图初始化

把想法落成基线：项目启动、环境自检与最小闭环初始化

本页结构

你本周要完成什么	2
为什么这页重要	3
本周真正要跑起来什么	3
这和后面几周的关系	3
今天要留下什么证据	3
这页和作业页的区别	3
本页和作业页的边界	3
作业页负责什么	3
实验页负责什么	3
你将得到什么	3
1. 可运行的服务环境	3
2. 健康检查结果	3
3. 最小 seed 数据	4
4. 一次 contract tests 结果	4
5. 一次 contract tests 结果	4
6. 一次 API 冒烟验证	4
7. 一份实验记录	4
实验总流程	4
操作步骤详解	5
Step 0 先把本周基线说清楚	5
为什么先做这一步	5
通过标准	5
留痕要求	5
Step 1 获取基线仓库	5
为什么先做这一步	5
通过标准	5
常见错误	6
留痕要求	6
Step 2 准备环境变量	6
为什么做	6
通过标准	6
常见错误	6
留痕要求	6
Step 3 启动 Docker Compose	6
为什么做	6
通过标准	7
常见错误	7
留痕要求	7

Step 4 执行健康检查	7
为什么做	7
通过标准	7
常见错误	7
留痕要求	7
Step 5 生成最小 seed 数据	8
为什么做	8
通过标准	8
常见错误	8
留痕要求	8
Step 6 执行 seed loader dry-run	8
为什么做	8
通过标准	8
常见错误	8
留痕要求	9
Step 7 运行 contract tests 并记录结果	9
为什么做	9
通过标准	9
常见错误	9
留痕要求	9
Step 8 执行 RAG 冒烟查询与 release 检查	9
为什么做	9
通过标准	9
常见错误	10
留痕要求	10
Step 9 输出实验记录与待补事项	10
推荐结构	10
最低要求	10
不要这样交	10
预期产出	10
环境结果	10
健康检查证据	11
数据与测试结果	11
接口与版本结果	11
实验记录文档	11
评分与验收	11
从本周到下周	11
提交前自检清单	11
课时、作业与实验回看入口	12

你本周要完成什么

这页实验不是再写一版蓝图，而是把 Week01 讲过的工程基线，真正落到仓库、服务、命令、自检和留痕上。你今天要完成的，不是“功能做得多”，而是先把一个 AI 项目推进到 **能启动、能检查、能留痕、能接到下一周** 的状态。

为什么这页重要

本周真正要跑起来什么

不是“完整 Copilot”，而是一条最小工程基线：

环境变量、Compose 服务、健康检查、最小 seed 数据、契约测试和实验记录。

这和后面几周的关系

Week01 先把起跑线立住。

Week02 才有条件把输入边界和数据契约写成机器可读定义，Week03 再去补 ingest 和入湖链路。

今天要留下什么证据

你需要留下健康检查结果、关键界面截图、最小 seed 生成结果、seed loader 报告、contract tests 结果、RAG 冒烟结果，以及本周待补事项。

这页和作业页的区别

作业页偏问题定义、蓝图、Done 和风险边界。

实验页偏启动环境、执行命令、自检通过、结果留痕。

! 一句话讲透

实验页负责把 Week01 的架构认知落成可执行基线；作业页负责把 Week01 的系统口径收成可评审文档。

两页相关，但不能互相替代。

本页和作业页的边界

⚠ Warning

作业页偏“想清楚”，实验页偏“跑起来并留痕”。

不要把实验截图直接当作业正文，也不要将作业写成一份项目启动手册。

作业页负责什么

- 问题定义
- 系统蓝图
- Done 标准
- 风险边界

实验页负责什么

- 项目骨架启动
- 环境健康检查
- 最小 seed 导入
- seed loader 与 contract tests 留痕

你将得到什么

1. 可运行的服务环境

通过 Docker Compose 拉起本地最小运行链路，不依赖宿主机 Python、PostgreSQL 或 MinIO。

2. 健康检查结果

确认 RAG API、Tool API、Dagster、MinIO、Phoenix 是否可访问。

3. 最小 seed 数据

生成第一批可重复使用的工单类种子数据。

4. 一次 contract tests 结果

确认 `seed manifest` 能被系统按契约识别，而不是只停留在“文件存在”。

5. 一次 contract tests 结果

从第一周开始建立“契约先行、测试留痕”的工程习惯。

6. 一次 API 冒烟验证

确认 Week01 的系统已经能返回带 `release_id` / `trace_id` 的最小响应。

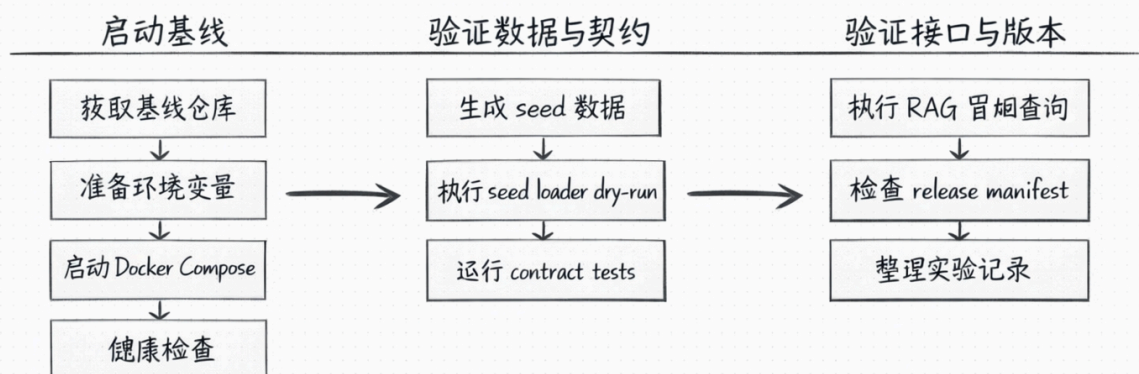
7. 一份实验记录

记录命令、截图、检查结果、失败项和修复动作。

实验总流程

Week01 实验总流程

先把项目基线拉起来，再验证数据、接口和留痕



目标: 可启动、可验证、可记录

i Note

下面的命令以课程主案例仓库为例。

你要学的不是“把 OmniSupport Copilot 特例背下来”，而是学会一条通用的 Week01 基线初始化流程。

! 实操口径来源

本页当前的命令顺序、检查项和验收口径，已对齐 OmniSupport Copilot 的 Week01 启动手册。如果后续仓库里的启动路径继续升级，默认应以这份 runbook 为准：

- [OmniSupport Copilot | runbooks/week01-startup.md](#)

操作步骤详解

Step 0 | 先把本周基线说清楚

Week01 的实验目标，不是做出“完整 Copilot”，而是把基础设施、目录、服务入口、最小 seed 数据、seed loader、contract tests、RAG 冒烟查询和 release 检查先拉通。只有这条最小链路通了，后面关于数据契约、采集与入湖、RAG API、Tool Layer、观测和发布，才有稳定落点。

为什么先做这一步

如果起跑线都不统一，团队后面很容易掉进“我的机器能跑，你的机器不能跑”的伪协作状态。

通过标准

- 你能说清这周实验要拉通哪些组件
- 你知道今天不追求完整业务闭环
- 你知道实验页和作业页分别产出什么

留痕要求

在实验记录开头写明：

- 本周实验目标
- 本周不做什么
- 你准备验证哪些服务和文件

Step 1 | 获取基线仓库

课程主案例的基线仓库在这里：

- [OmniSupport Copilot](#)

先把仓库拉到本地，再开始后面的环境准备和服务启动。

这一步的意义，不是“拿到一份代码”，而是统一所有人后续操作的起点。

```
git clone https://github.com/dataPro-lgtm/omnisupport-copilot.git
cd omnisupport-copilot
```

为什么先做这一步

- 实验页后面的命令都默认你已经在课程主案例仓库里
- 先把仓库拿到本地，才能统一目录、命令和留痕口径
- 这一周强调的是“统一起跑线”，不是各自在脑中想象项目结构

通过标准

- 你已经把仓库 clone 到本地
- 你已经进入仓库根目录

- 你知道后续所有命令都应该在这个工作目录里执行

常见错误

- 还没获取仓库就直接执行后面的命令
- 在错误目录里运行 Compose 或测试命令
- 只下载 ZIP，不建立可更新的本地 Git 工作副本

留痕要求

- 记录本地仓库目录
- 在实验记录里写清你使用的代码版本或拉取时间

Step 2 | 准备环境变量

```
cp infra/env/.env.example infra/env/.env.local
```

为什么做

- 机密配置不能写死在代码里
- 本地 / 测试 / 线上配置必须分离
- `.env.example` 是团队协作契约，不是装饰文件
- Week01 默认走 Docker-only 路线，所以这一步的重点是 `.env.local`，不是先折腾宿主机依赖

通过标准

- `infra/env/.env.local` 已生成
- 你知道哪些值必须填写、哪些可以暂时留空
- 至少能区分“今天可先跑部分能力”和“必须依赖密钥的能力”
- 你知道 Week01 中 `ANTHROPIC_API_KEY` 可以先留空，系统会返回 fallback 响应

常见错误

- 直接改 `.env.example`
- 把个人密钥写进仓库
- 不区分必填项和可延后项
- 以为必须先装本地 Python 才能继续

留痕要求

- 截图或记录 `.env.local` 已创建
- 在实验记录里写清今天缺失哪些密钥、会影响哪一步

Step 3 | 启动 Docker Compose

```
docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml up -d --build
```

为什么做

- Compose 用来统一服务、网络、卷和依赖
- 它降低团队环境差异
- 它让“最小闭环”可以被重复拉起，而不是只活在某个人电脑里

通过标准

- Compose 没有关键启动错误
- 你能看到 `postgres / minio / rag_api / tool_api / dagster / otel_collector / phoenix` 为 Up
- 你知道 `minio_init` 是一次性初始化容器，不需要一直常驻
- 至少能进入下一步健康检查

常见错误

- 端口占用
- 镜像未拉取完整
- 环境变量缺失导致部分服务启动失败
- 误以为宿主机必须先启动本地 PostgreSQL

留痕要求

- 保存 `docker compose ps` 或等价结果
- 如果失败，记录失败服务名和报错摘要

Step 4 | 执行健康检查

```
curl -s http://localhost:8000/health # RAG API
curl -s http://localhost:8001/health # Tool API
```

```
# 在浏览器访问：
# http://localhost:3000 Dagster UI
# http://localhost:9001 MinIO Console
# http://localhost:6006 Phoenix
```

为什么做

健康检查不是“进程活着”，而是“关键能力可达”。

Week01 就暴露 Phoenix，不是超前，而是在告诉你：可观测性不是 Week12 才开始。

通过标准

- `RAG API` 与 `Tool API` 返回健康状态
- `Dagster / MinIO / Phoenix` 可以打开
- 你能确认 `MinIO Console` 可登录，并看到初始化后的 `buckets`
- 没有明显启动报错或空白页

常见错误

- 服务进程活着，但 `health` 接口不通
- UI 能打开，但底层依赖没连上
- 只记“能开页面”，没记返回结果

留痕要求

- 至少保留 3 张关键截图
- 记录每个检查点“通过 / 未通过 / 待补”

Step 5 | 生成最小 seed 数据

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
python data/synthetic_generators/ticket_simulator.py --count 500 \
--output data/canonization/tickets/tickets-seed-001.jsonl
```

为什么做

- 先把工程基线跑通
- 给后续测试、回归和演示提供可重复输入
- 降低一开始直接接真实数据的权限与合规压力

通过标准

- 输出文件成功生成
- 你知道这份 seed 数据位于哪里
- 你能解释它后面会如何进入 canonization / ingest 链路

常见错误

- 只跑命令，不检查输出路径
- 把 seed 当成“假装生产数据”，忽略它的工程意义
- 生成成功却不记录文件名和位置
- 在宿主机直接跑 Python 命令，偏离课程默认路径

留痕要求

- 记录输出文件路径
- 截图或保留一小段数据样例

Step 6 | 执行 seed loader dry-run

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
python -m pipelines.ingestion.seed_loader \
--manifest-dir data/seed_manifests
```

为什么做

- Week01 不只验证“文件在不在”，而是要验证 manifest 能否被系统按契约识别
- 这一步在 Week02 之前先把输入边界跑通
- 让 seed 数据、manifest 和 ingest 口径从第一周开始对齐

通过标准

- 输出中能看到 manifest 被成功加载
- 没有 REJECTED 条目
- 你能说明今天验证的是“契约和入口”，不是完整 ingest 入湖

常见错误

- 只生成 seed 文件，不继续验证 manifest
- 把 dry-run 当成真实导入，混淆 Week01 与 Week02 的边界

- 出现 reject 但不记录是 schema 问题还是路径问题

留痕要求

- 保存 seed loader 报告摘要
- 记录 valid / rejected 数量
- 如果失败，记录失败的是哪份 manifest

Step 7 | 运行 contract tests 并记录结果

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    pytest tests/contract/ -v
```

为什么做

- 从第一周开始建立“契约先行、测试留痕”的工程意识
- 这一步不是追求测试多，而是先验证系统边界有没有被写下来

通过标准

- 测试能成功运行
- 你知道哪些测试通过、哪些失败
- 你能解释失败意味着哪一层边界还没补齐

常见错误

- 只记“跑过了”，不记结果
- 测试失败后直接跳过，不记录原因
- 把契约测试理解成最后才需要的补充工作
- 还沿用宿主机 `pip install -e ".[dev]"` 的旧路径

留痕要求

- 保存测试命令输出摘要
- 如果失败，记录失败项、可能原因和待补动作

Step 8 | 执行 RAG 冒烟查询与 release 检查

```
curl -s -X POST http://localhost:8000/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"query": "如何配置 Northstar Workspace SS0?"}'

curl -s http://localhost:8000/api/v1/admin/release
```

为什么做

- Week01 要验证的不只是“服务活着”，还要验证接口已经带上 `release_id / trace_id`
- 就算今天没填 `ANTHROPIC_API_KEY`，也应该能看到系统返回 fallback 响应
- release manifest 是后面评测、回滚和治理的基线，不是 Week14 才临时补的

通过标准

- `/query` 能返回最小 JSON 响应

- 响应里能看到 `release_id` 和 `trace_id`
- `/api/v1/admin/release` 能返回当前 `release` 信息

常见错误

- 只看 `health`，不继续验证用户侧接口
- 没填 `API key` 就误判系统“完全不能用”
- 忽略 `release manifest`，只把它当作后期治理内容

留痕要求

- 保存一次 `query` 响应摘要
- 记录当前 `release_id`
- 如果是 `fallback` 响应，要在实验记录里写明原因

Step 9 | 输出实验记录与待补事项

这一页实验最后要交的，不是一段总结话术，而是一份能被助教复核的实验记录。它至少要让人看懂：你做了什么、跑成了什么、卡在哪里、下一周要接什么。

推荐结构

1. 环境信息
2. 执行命令
3. 关键截图
4. 健康检查结果
5. `seed` 数据生成结果
6. `seed loader` 报告
7. `contract tests` 结果
8. `query` / `release` 响应摘要
9. 失败项 / 修复动作 / 待补事项

最低要求

- 关键命令出现过
- 关键结果留痕了
- 失败项被明确写出来
- 已说明如何进入 `Week02`

不要这样交

- 只交几张截图，没有上下文
- 只写“都成功了”，没有结果证据
- 把实验记录写成一篇项目宣传稿

预期产出

环境结果

- `Compose` 启动结果
- 关键服务状态
- 失败服务说明

健康检查证据

- API 返回结果
- Dagster / MinIO / Phoenix 截图
- 通过 / 未通过标记

数据与测试结果

- seed 文件路径
- seed loader 结果
- contract tests 结果
- 失败项记录

接口与版本结果

- /query 响应摘要
- `release_id`
- fallback / 正常生成说明

实验记录文档

- 命令
- 截图
- 问题
- 修复动作
- 下周待补清单

评分与验收

维度	分值	验收重点
启动完整性	25	是否完成环境准备、Compose 启动和关键服务检查
数据与契约验证	25	是否完成 seed、seed loader 和 contract tests
证据留痕	25	是否有命令、截图、query / release 结果和失败项记录
与下周承接性	25	是否明确今天的结果如何流入 Week02

从本周到下周

! Important

今天跑起来的只是 baseline，不是完整系统。

下周要继续补的是：输入边界、数据契约和字段定义。

也就是说，今天先把 `contracts/`、seed 数据、服务入口和实验记录放好；下周再把“数据怎样稳定进入系统”补齐。

提交前自检清单

- ☐ 我已经获取课程主案例仓库，并在正确目录中执行命令
- ☐ 我已经生成并检查 `.env.local`
- ☐ 我已经启动 Compose，并记录关键服务状态
- ☐ 我已经完成 RAG API / Tool API / Dagster / MinIO / Phoenix 的最小健康检查
- ☐ 我已经生成 seed 数据，并记录输出文件

- ☐ 我已经执行 seed loader dry-run, 并记录 valid / rejected 结果
- ☐ 我已经运行 contract tests, 并记录结果
- ☐ 我已经执行 RAG 冒烟查询, 并记录 `release_id` / `trace_id`
- ☐ 我已经保存关键截图和命令输出
- ☐ 我已经写清失败项、修复动作和待补事项
- ☐ 我知道这些结果会如何进入 Week02

课时、作业与实验回看入口

如果你在实验过程中卡住, 优先回看这些页面:

- [课时 1: 为什么你的 AI Demo 一上生产就翻车?](#)
- [课时 2: 知识库问答 + 工单联动怎么定义 Done](#)
- [课时 3: 别一上来就写 RAG: 企业级 AI 到底该走哪条数据工程路线](#)
- [课时 4: 开营就把工程底座搭起来: 项目骨架、风险边界与落地蓝图一次定清](#)
- [作业页: 把 Demo 思维收口为工程蓝图](#)